



DSA PROJECT

Title:

FireBoy WaterGirl (Game)

Submitted to :

- **Instructor :** Ma'am Humera Niaz
- **Course :** Data Structures (CSC-211)

Submitted by:

- Sannan Ahmad (SP25-BSE-118)
- Areej Raza Ali (SP25-BSE-028)
- AsadUllah Riaz (SP25-BSE-031)
- M.Hassan (SP25-BSE-088)

Date of Submission:

17 May, 2026

[CLICK HERE TO ACCESS THE SOURCE CODE](#)

Introduction and Features of the Project.....	3
Introduction.....	3
Key Features.....	3
Gameplay Features.....	3
Technical / DSA Features.....	3
UI / Audio Features.....	4
Conclusions, Findings, Challenges Faced, and Potential Future Enhancements.....	4
Conclusions and Findings.....	4
Challenges Faced.....	5
Potential Future Enhancements.....	6
Contribution of Each Group Member.....	6
Output / Screenshots of Project Execution	6
Screenshot 1 — Splash Screen.....	6
Screenshot 2 — Name Entry Screen	7
Screenshot 3 — Level 1 :	8
Screenshot 5 — Level 4:.....	9

Introduction and Features of the Project

Introduction

Fireboy and Watergirl is a two-player cooperative puzzle-platformer game developed in C++ using the Qt framework. Inspired by the classic browser-based Flash game of the same name, this project recreates the core gameplay experience as a fully native desktop application. The game features two characters — Fireboy, who is immune to lava but harmed by water, and Watergirl, who is immune to water but harmed by lava — navigating through a series of increasingly challenging levels to reach their respective exit doors.

The project was built as part of a Data Structures and Algorithms course, with the deliberate goal of embedding real, meaningful DSA implementations into the game's core systems rather than using them superficially. Every data structure present in the codebase directly drives a gameplay feature.

The application is built using Qt's widget, multimedia, and GUI modules with a C++17 standard, and uses a QMake project file for building. The game loop runs at approximately 60 frames per second using a 16ms Qt timer.

Key Features

Gameplay Features

- Two-player cooperative platformer with Fireboy (Arrow Keys) and Watergirl (WASD)
- Five handcrafted levels with increasing difficulty, each with a unique background theme (cave, forest, ruins)
- Gem collection system — each player collects their respective coloured gems for score
- Lives system — players have 3 lives per run; losing all lives ends the game
- Hazard tiles — lava (kills Fireboy), water (kills Watergirl), and poison (kills both)
- Exit door system — Fireboy and Watergirl must both reach their respective doors simultaneously to complete a level
- Buttons and Gates — pressure-sensitive buttons toggle coloured gates to open/close paths
- Conveyor Belts — moving floor tiles that push players in a direction
- Teleportation Portals — matching portal pads that transport players across the level
- Hint System — press 'H' to highlight the shortest path to the nearest gem using Dijkstra's algorithm

Technical / DSA Features

- Doubly Linked List for level navigation (move forward/backward through levels)

- Undo/Redo System using a Doubly Linked List of game state snapshots (press U to undo, R to redo; snapshots taken every 500ms)
- Priority Queue (Min-Heap) for game event processing
- Min-Heap for finding the nearest uncollected gem to each player
- Dijkstra's Weighted Pathfinding Algorithm for computing the optimal hint path on a grid
- Quick Sort for sorting the leaderboard scores in descending order
- Binary Search for looking up scores on the leaderboard
- Hash Maps (Direct-Address Tables) for O(1) gate and teleport lookups by ID
- Binary Search Tree (BST) for tile data storage in each level
- Singly Linked List for tracking gem collection trail

UI / Audio Features

- Splash screen on startup
- Main menu with Play, Leaderboard, and Quit options
- Name entry screen before gameplay begins
- In-game HUD displaying current score, lives, and level number
- Leaderboard screen displaying top scores loaded from file and sorted via Quick Sort
- Full sound effects: background music, jump sounds (separate for Fireboy and Watergirl), gem collection, death, win
- Custom QSS stylesheets for consistent visual theming across all screens
- Undo/Redo visual flash feedback on screen

Conclusions, Findings, Challenges Faced, and Potential Future Enhancements

Conclusions and Findings

This project successfully demonstrates the practical application of Data Structures and Algorithms in a real, interactive software product. Rather than implementing DSA as isolated academic exercises, each structure was chosen specifically because it solved a concrete engineering problem within the game:

- The **Doubly Linked** List proved ideal for level management because levels are traversed sequentially in both directions and the game always needs access to the current node.
- The Undo/Redo system showed how a **doubly linked list** of state snapshots can implement time-reversal mechanics efficiently without storing the entire frame history.
- Dijkstra's algorithm on a grid-based map produced accurate, **weighted shortest-path** hints that account for different terrain costs, demonstrating that graph algorithms apply naturally to tile-based games.

- The **Min-Heap gem finder** provided an efficient way to always identify the closest uncollected gem without scanning the entire map every frame.
- The **Priority Queue** for event processing showed how events like button presses and gate toggles can be scheduled and handled in a controlled, ordered manner rather than being processed immediately and unsafely.
- **Quick Sort** on the leaderboard confirmed that even with a small dataset, using the correct algorithm provides clean, consistent results and is significantly more extensible than manual sorting.
- The **Hash Maps** for gates and teleports demonstrated $O(1)$ lookup performance, which is critical in a 60fps game loop where even small inefficiencies accumulate across thousands of calls per second.

Overall, the project reinforced the understanding that data structures are not abstract concepts but are the backbone of any performant, well-organised software system.

Challenges Faced

- **Qt Multimedia Compatibility:** Sound playback using QSoundEffect behaved differently across Qt versions, requiring conditional compilation with `#ifdef` Qt version checks to ensure compatibility on different machines.
- **Undo/Redo State Complexity:** Designing the GameState snapshot to be deep-copyable while remaining efficient was non-trivial. Capturing too much state caused memory bloat; capturing too little caused incomplete undo behaviour. Finding the right balance required multiple iterations.
- **Conveyor Belt Physics Interaction:** Integrating conveyor belt velocity into the existing collision resolution pipeline required careful ordering. The conveyor force had to be applied before collision checks, not after, to avoid players clipping through walls.
- **Teleport Gem Re-initialisation Bug:** A potential bug was identified where a player teleporting through a portal and back might have their gem count reset to zero due to level state being re-evaluated. Safeguards were put in place to prevent gem state from being overwritten on teleport.
- **Spawn Position Bug in Level 3:** After all lives were lost in Level 3, both Fireboy and Watergirl were incorrectly spawning at the same position (Fireboy's spawn point). This was resolved by ensuring each character's spawn coordinates are loaded independently on respawn.
- **Build Environment Portability:** Ensuring the project compiled and ran correctly on both Windows and Linux required platform-specific run scripts (`run.bat`, `run.ps1`, `run_linux.sh`) and careful management of asset paths.

Potential Future Enhancements

- **Additional Levels:** The level architecture (Doubly Linked List of LevelData) is fully extensible. New levels can be appended to the list without any changes to the engine code, making it straightforward to add 5–10 more levels.
- **Moving Platforms:** The infrastructure for moving platforms was partially built but removed to simplify the codebase. Re-introducing moving tiles as a first-class game mechanic would add significant gameplay variety.
- **Animated Sprites:** The current game uses static tile images. Replacing them with sprite sheet animations (walking cycles, idle animations) would substantially improve the visual quality of the game.
- **AI-Controlled Second Player:** Implementing a simple AI agent to control one of the characters (using the existing Dijkstra pathfinding infrastructure) would allow single-player mode without requiring a second human player.
- **Checkpoint System:** An earlier design considered checkpoints as an alternative to undo/redo. Both could coexist — Undo for fine-grained reversal and Checkpoints for coarse-grained mid-level saves.
- **Settings Menu:** Adding a settings screen for volume control, key rebinding, and resolution options would improve accessibility and user experience.

Contribution of Each Group Member

Member Name	Contributions
Sannan Ahmad (SP25-BSE-118)	Led the project & code implementation, contributed to tech stack selection, and shaped the overall game flow.
Areej Raza Ali (SP25-BSE—028)	Designed the UI (GameRenderer.cpp) interface, dealt with the mechanics of the game and originated the core game idea.
AsadUllah Riaz (SP25-BSE-031)	Provided ideas for new data structures and handled debugging and testing.
M.Hassan (SP25-BSE-088)	Provided valuable suggestions and assisted in quality checks.

Output / Screenshots of Project Execution

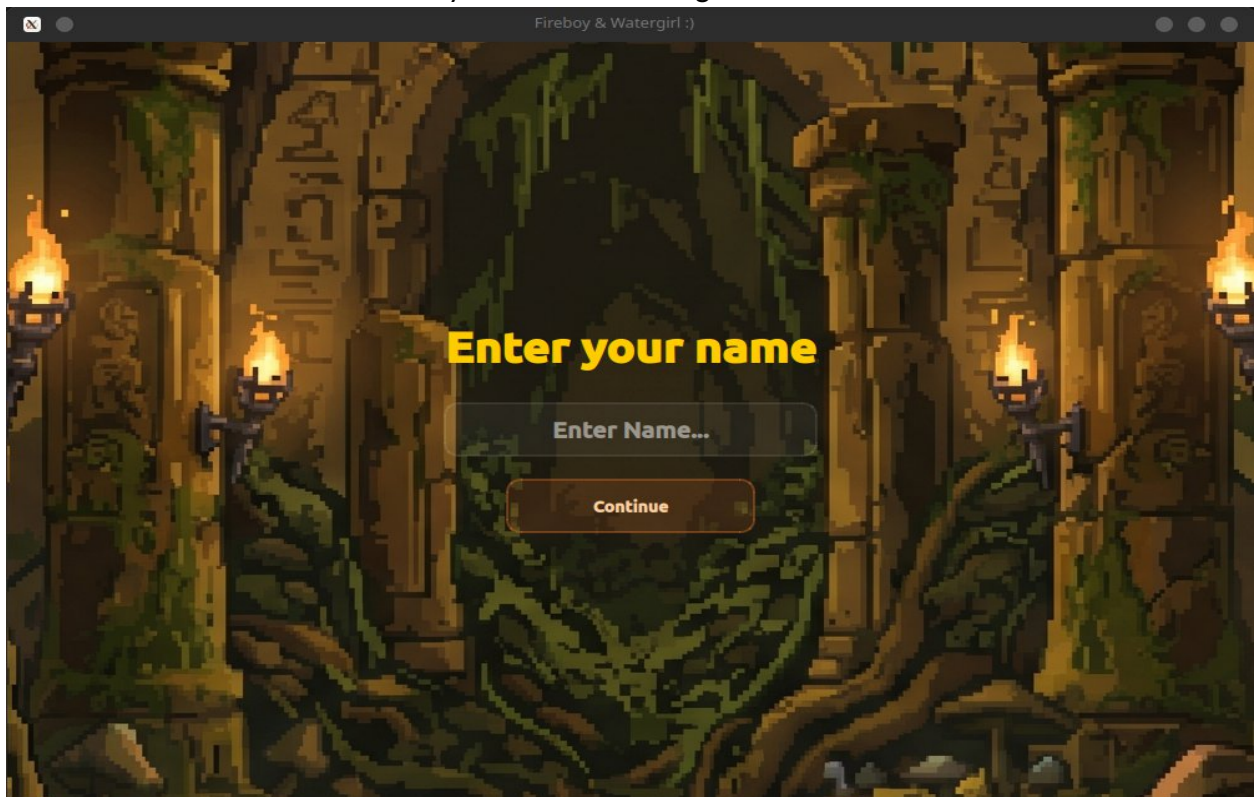
Screenshot 1 — Splash Screen

The splash screen is displayed for 3 seconds when the application launches, while the game window initialises in the background.

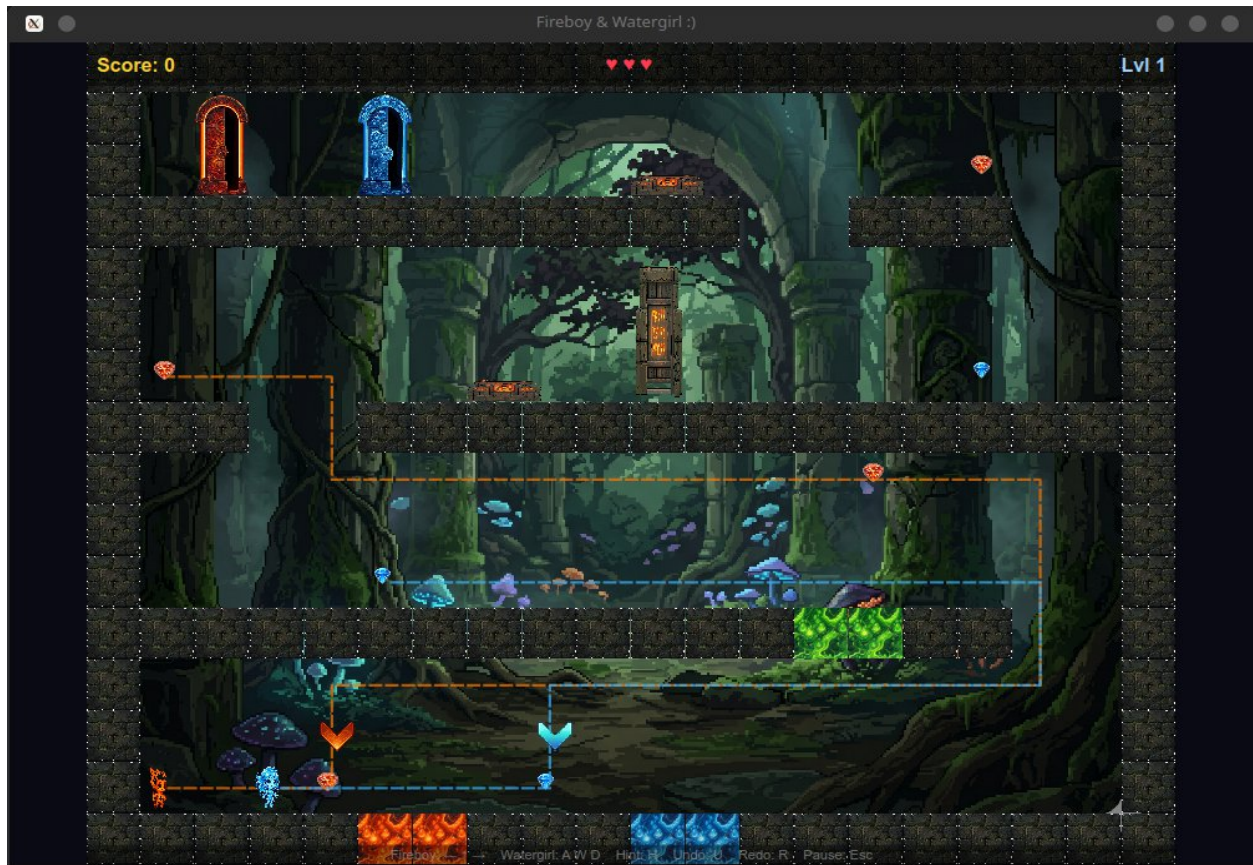


Screenshot 2 — Name Entry Screen

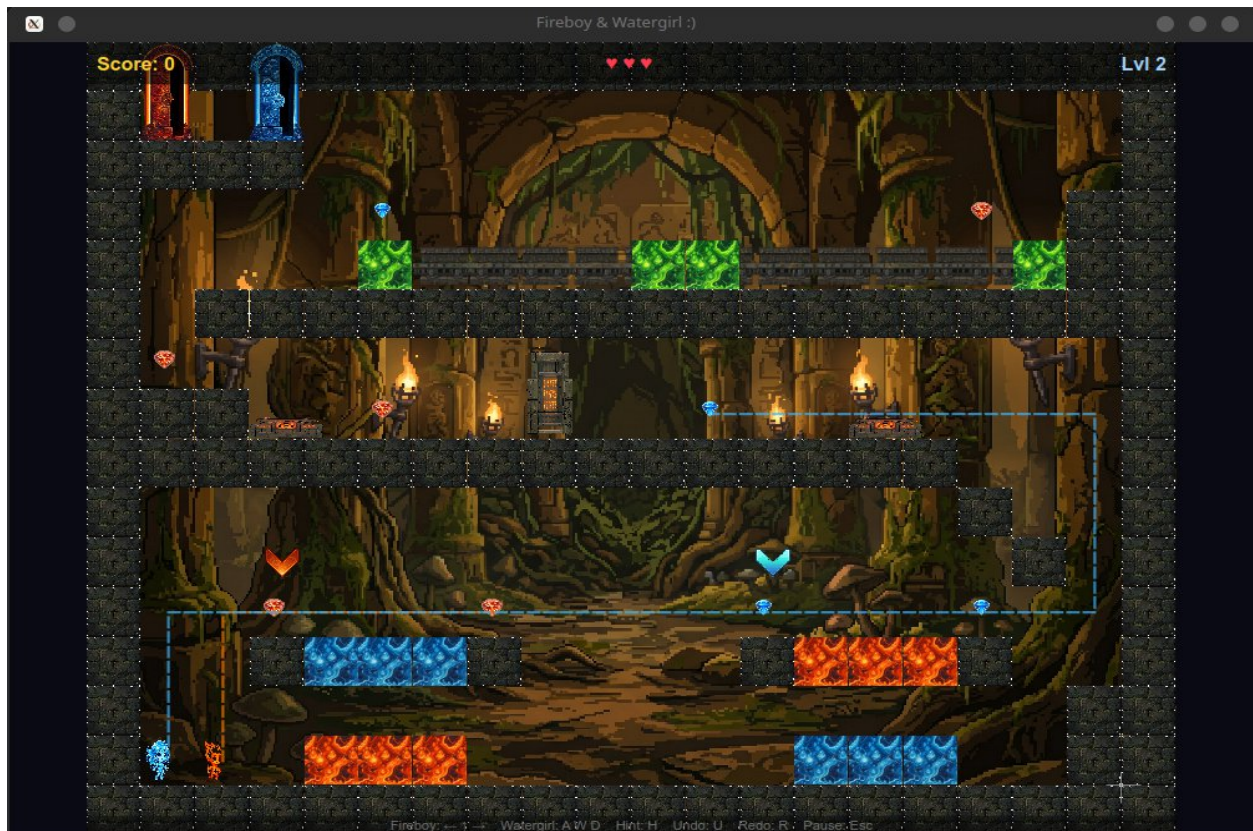
Before the game begins, players are prompted to enter their names. These names are stored and used for the leaderboard entry at the end of the game.



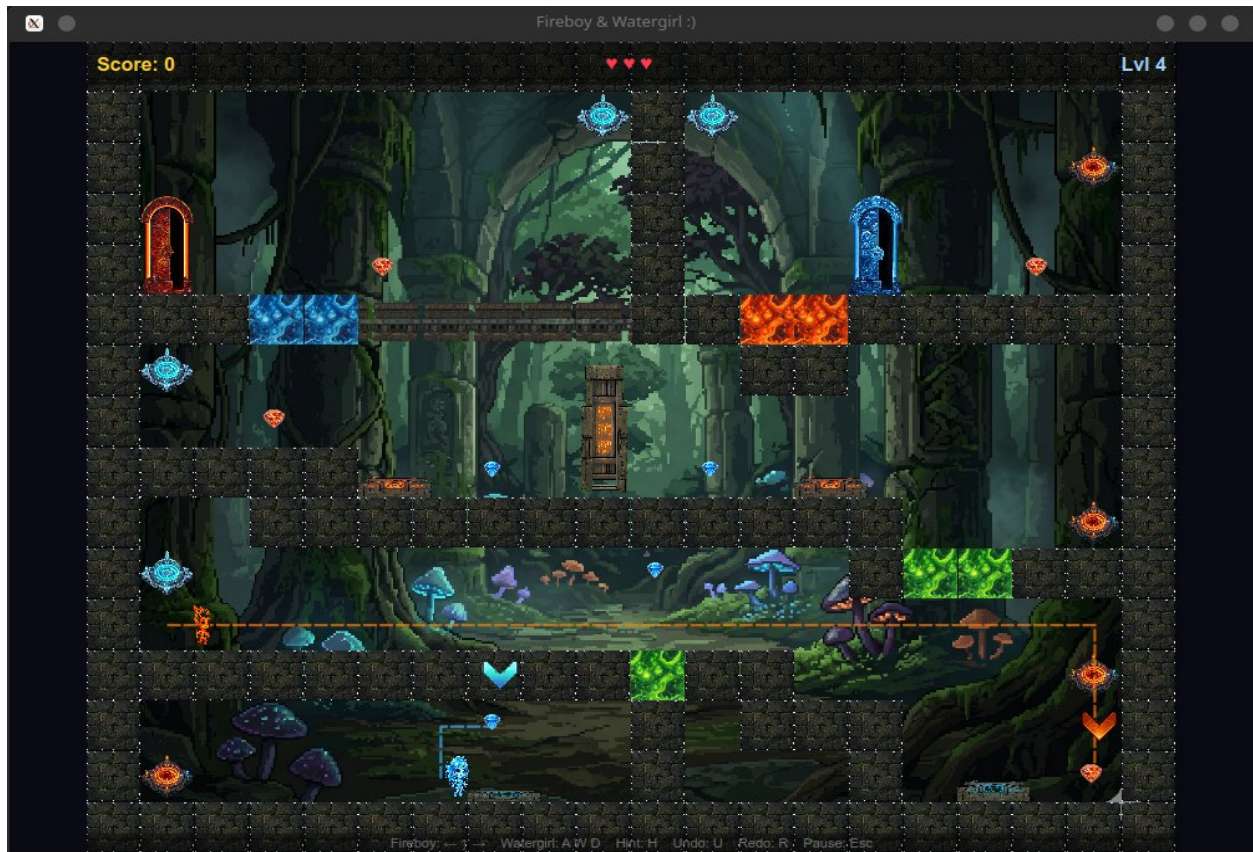
Screenshot 3 — Level 1 :



Screenshot 4 — Level 2:



Screenshot 5 — Level 4:



Screenshot 6 — Leaderboard Screen :



FIN-!